

UNIT II

Software Requirements: Functional and non-functional requirements, User requirements, System requirements, Interface specification, The software requirements document.

Requirements engineering process: Feasibility studies, Requirements elicitation and analysis, Requirements validation, Requirements management, RTM.

In software engineering, **requirements** refer to the specifications that define what a software system should do and how it should perform. They serve as a blueprint for both the development process and the final product, guiding design, implementation, and testing.

Types of Requirements:- There are 5 types of S/W Requirements.

1. Functional Requirements
2. Non-Functional Requirements
3. User Requirements
4. System Requirements
5. Interface Requirements

•Types of Software Requirements

1. Functional Requirements:

- **Definition:** Describe specific functionalities that the system must support.
- **Examples:**
 - User authentication and authorization.
 - Data entry and validation.
 - Reporting features.
- **Documentation:** Often captured through use cases, user stories, or requirements specifications.

2. Non-Functional Requirements:

- **Definition:** Outline criteria that judge the system's operation rather than specific behaviors.
- **Categories:**
 - **Performance:** Response time, throughput, and latency.
 - **Usability:** User experience, accessibility standards.
 - **Reliability:** Availability, fault tolerance.
 - **Security:** Data protection, compliance with regulations.
- **Documentation:** Typically documented in separate sections of requirements specifications.

3. User Requirements:

- **Definition:** Specific needs and expectations of the end-users.
- **Focus:** User interactions and experience with the software.
- **Documentation:** Often represented as user stories or scenarios.

4. System Requirements:

- **Definition:** Technical specifications that describe the hardware and software environment.
- **Types:**
 - **Hardware Requirements:** CPU, memory, storage specifications.
 - **Software Requirements:** Operating systems, libraries, and frameworks needed.

5. Interface Requirements:

- **Definition:** Describe how the software will interact with other systems or components.
- **Types:**
 - **User Interface (UI):** Layout, navigation, and accessibility guidelines.
 - **Application Programming Interfaces (APIs):** Specifications for external integrations.

Functional requirements

- **Functional requirements** are a crucial aspect of software engineering, detailing what the system should do and how it should behave in various scenarios. Here's a deeper dive into functional requirements:

- **Definition**

- **Functional Requirements** specify the expected behavior of a system, outlining specific functions, features, and interactions that the software must support. They describe what the system should accomplish and often serve as the foundation for system design, implementation, and testing.

- **Key Characteristics**

1. **Specificity:** Clearly define specific actions or functions the software must perform.
2. **Testability:** Should be measurable or verifiable through testing.
3. **User-Centric:** Focus on what the users need from the system.
4. **Traceability:** Each functional requirement should be linked to a business or user need.

•Components of Functional Requirements

1. Use Cases: Scenarios that describe how users interact with the system.

- Example: “A user can log in using their email and password.”

2. User Stories: Short, simple descriptions of a feature from the perspective of the end-user.

Example: “As a user, I want to reset my password so that I can regain access to my account.”

3. System Functions: Specific capabilities or tasks the system must perform.

- Example: “The system shall allow users to search for products by name or category.”

4. Business Rules: Constraints or conditions that dictate how the system operates.

- Example: “A user cannot place an order without providing a shipping address.”

5. Data Management: Data handling, storage, and retrieval requirements.

- Example: “The system shall store user profiles securely and retrieve them upon request.”

Examples of Functional Requirements

1. Authentication:

- Users must be able to register and create an account.
- Users must be able to log in using a username and password.

2. User Management:

- Admin users can create, update, and delete user accounts.
- Users can update their profile information.

3. Search Functionality:

- Users can search for products by various filters (category, price range, ratings).
- The system must display search results within 2 seconds.

4. Reporting:

- The system shall generate monthly sales reports for analysis.
- Users can export reports in PDF and Excel formats.

5. Notifications:

- Users shall receive email notifications for important updates (e.g., order confirmation).
- The system must alert users when their account settings change.

- Gathering Functional Requirements**

- 1. Stakeholder Interviews:** Discuss needs and expectations with users and stakeholders.

- 2. Workshops:** Collaborative sessions to identify and prioritize requirements.

- 3. Prototyping:** Create mockups to visualize functionality and gather feedback.

- 4. Review Existing Systems:** Analyze current systems for functionality that can be improved or replicated.

Non-functional requirements

- Definition**

- Non-Functional Requirements** describe the characteristics of the system that impact user experience, system performance, and overall quality. They encompass a wide range of criteria that govern system behavior and performance, rather than specific functionalities.

- Key Categories of Non-Functional Requirements**

- 1. Performance:**

- **Response Time:** The time taken for the system to respond to a user action.
 - Example: “The system shall respond to user queries within 2 seconds.”
- **Throughput:** The number of transactions or requests the system can handle in a given time frame.
 - Example: “The system shall support 1000 concurrent users without performance degradation.”

- 2. Scalability:**

- The ability of the system to handle increased loads without performance loss.
 - Example: “The system shall scale to accommodate 10,000 users.”

3. Reliability:

- The likelihood that the system will perform its required functions under stated conditions for a specified period.
 - Example: “The system shall have 99.9% uptime over a year.”

4. Availability:

- The degree to which the system is operational and accessible when required for use.
 - Example: “The system shall be available 24/7.”

5. Security:

- Measures to protect the system from unauthorized access and data breaches.
 - Example: “User passwords must be stored using encryption.”

6. Usability:

- The ease with which users can learn and use the system.
 - Example: “New users shall be able to complete basic tasks within 10 minutes of first use.”

7. Maintainability:

- The ease with which the system can be modified to fix defects, improve performance, or adapt to a changing environment.
 - Example: “The system shall allow updates without downtime.”

8. Portability:

- The ability of the system to be transferred from one environment to another.
 - Example: “The application shall run on both iOS and Android platforms.”

9. Compliance:

- Adherence to laws, regulations, and standards.
 - Example: “The system shall comply with GDPR for data protection.”

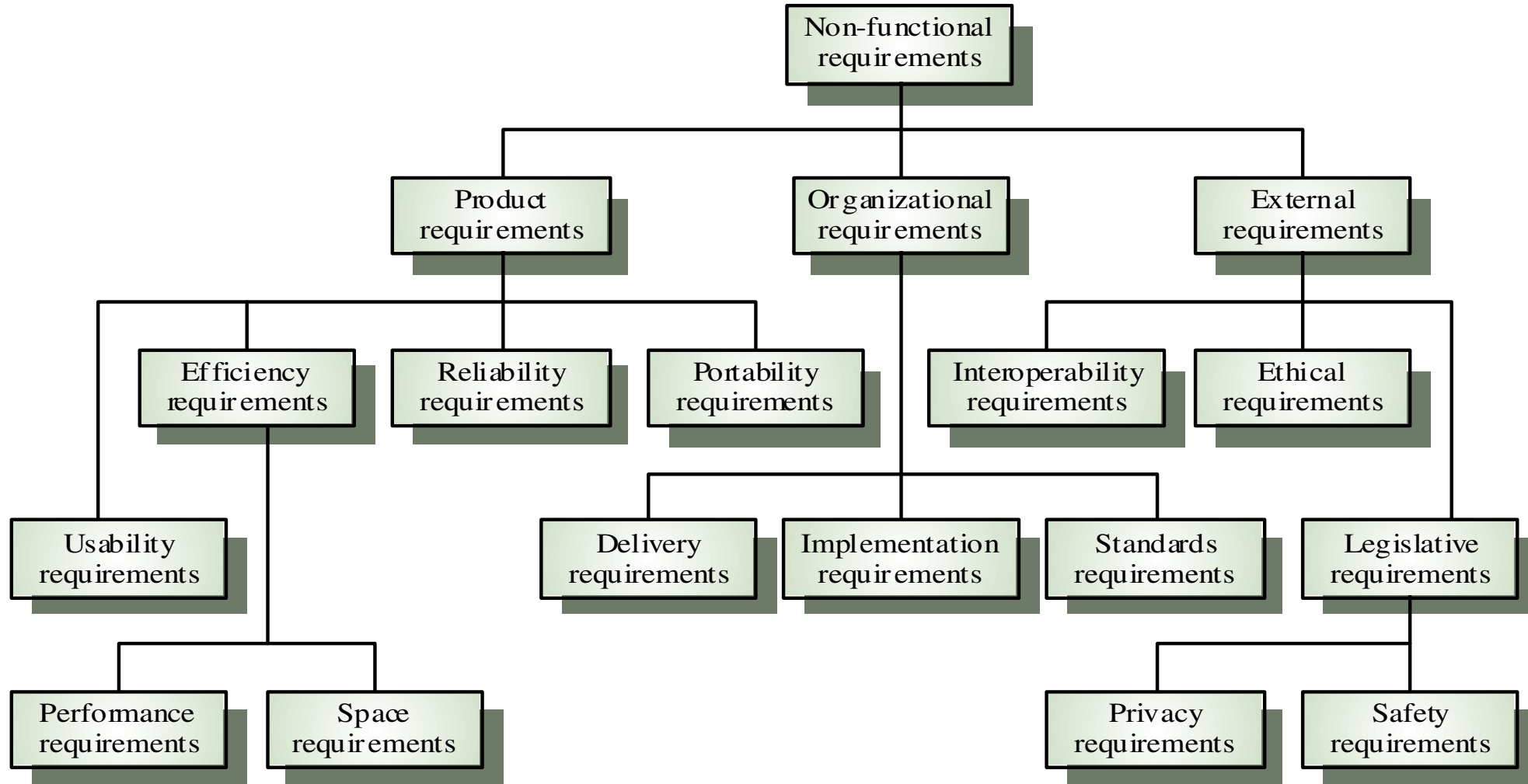
•Importance of Non-Functional Requirements

- **User Satisfaction:** They significantly impact user experience and satisfaction.
- **System Longevity:** High-quality NFRs can enhance system maintainability and scalability.
- **Competitive Advantage:** Meeting high standards for performance and usability can differentiate a product in the market.

•**Gathering Non-Functional Requirements**

- 1. Stakeholder Interviews:** Discuss expectations and quality attributes with users and stakeholders.
- 2. Surveys:** Collect feedback on user preferences regarding performance and usability.
- 3. Benchmarking:** Analyze similar systems to identify industry standards and user expectations.
- 4. Prototyping:** Develop prototypes to test and gather feedback on non-functional attributes.

Non-functional requirement types



User Requirements

User requirements in software engineering focus on the needs and expectations of the end-users who will interact with the system. These requirements are crucial for creating software that is user-centered, ensuring that the final product meets the actual demands of its users. Here's an overview of user requirements, their significance, and how to gather and document them effectively.

- Definition**

- User Requirements** are statements that describe what users need from a software system, emphasizing their interactions and experiences. These requirements guide the design, development, and testing phases, ensuring the software fulfills user needs.

- Characteristics of User Requirements**

- **User-Centric:** Focused on the needs and goals of the end-user.
- **Clear and Concise:** Written in simple language that is easy to understand.
- **Testable:** Can be verified through testing to ensure the software meets user expectations.
- **Prioritized:** Ranked based on importance and urgency to ensure critical features are developed first.

Types of User Requirements

Functional User Requirements:

- Describe specific functionalities that users expect.
- Example: “Users must be able to reset their passwords via email.”

Non-Functional User Requirements:

- Describe the quality attributes of the system.
- Example: “The application should load within 3 seconds on a standard internet connection.”

User Interface Requirements:

- Specify how the system should present information to users.
- Example: “The dashboard should display user statistics in a graphical format.”

Importance of User Requirements

- **User Satisfaction:** Helps ensure that the software meets user needs, leading to higher satisfaction.
- **Reduced Rework:** Clearly defined user requirements minimize misunderstandings, reducing the need for rework later in the development process.
- **Guidance for Design and Testing:** Provides a foundation for creating design specifications and test cases.

Gathering User Requirements

Interviews: Conduct one-on-one or group discussions with users to understand their needs.

Surveys/Questionnaires: Use structured forms to gather input from a larger user base.

Workshops: Facilitate collaborative sessions to elicit requirements from multiple stakeholders.

Observations: Watch users interact with existing systems to identify pain points and needs.

Prototyping: Create mockups or prototypes to visualize functionality and gather user feedback.

Documenting User Requirements

User Stories: Short, simple descriptions of features from the user's perspective.

- Format: "As a [user role], I want [goal] so that [reason]."
- Example: "As a shopper, I want to filter products by price so that I can find affordable options."

Use Cases: Detailed scenarios that describe how users will interact with the system.

- Include actors, preconditions, and postconditions.
- Example: A use case for logging in might describe the steps a user takes to enter credentials and gain access.

Requirements Specification Document: A comprehensive document that includes all user requirements, organized by categories, with clear descriptions and acceptance criteria.

System requirements

- **System requirements** in software engineering define the necessary specifications for a system's hardware and software components. They serve as a foundation for system design, development, and testing. Here's a detailed overview of system requirements, including their types, importance, and best practices for gathering and documenting them.

- **Definition**

- **System Requirements** are the essential specifications that outline the necessary features, capabilities, and constraints of a system. They encompass the technical and operational aspects that the software must meet to function correctly within its intended environment.

Types of System Requirements

Functional Requirements:

- Describe specific behaviors and functions the system must support.
- Example: “The system shall allow users to create and manage their profiles.”

Non-Functional Requirements:

- Define the quality attributes of the system, such as performance, security, and usability.
- Example: “The system shall process user requests within 2 seconds.”

Hardware Requirements:

- Specify the physical components needed for the software to run effectively.
- Example: “The server must have at least 16 GB of RAM and a 500 GB SSD.”

Software Requirements:

- Detail the software environments, tools, and frameworks needed for development and deployment.
- Example: “The application shall run on Java 11 and utilize the Spring Framework.”

Interface Requirements:

- Describe how the system will interact with other systems or components.
- Example: “The system shall integrate with the payment gateway API for transaction processing.”

Environmental Requirements:

- Specify the conditions under which the system must operate.
- Example: “The application must function in both online and offline modes.”

Security Requirements:

- Define the measures needed to protect the system from threats and unauthorized access.
- Example: “All user data must be encrypted during transmission.”

Importance of System Requirements

- **Guidance for Development:** Provides a clear framework for developers to follow during implementation.
- **Alignment with User Needs:** Ensures that the technical specifications align with the functional and user requirements.
- **Basis for Testing:** Serves as a reference point for creating test cases and validation criteria.
- **Risk Management:** Helps identify potential technical risks and constraints early in the project.

Gathering System Requirements

Stakeholder Interviews: Engage with users, clients, and other stakeholders to gather insights on their needs and expectations.

Surveys/Questionnaires: Use structured surveys to collect information from a larger audience.

Workshops: Facilitate collaborative sessions to discuss and prioritize requirements.

Document Analysis: Review existing documentation and systems to identify relevant requirements and constraints.

Prototyping: Develop prototypes to visualize system behavior and gather feedback on technical requirements.

Documenting System Requirements

Requirements Specification Document: A comprehensive document that includes all system requirements, organized into categories with clear descriptions and acceptance criteria.

Use Cases: Detailed scenarios that describe how the system will behave in specific situations, including interactions with users and other systems.

Requirement Traceability Matrix (RTM): A tool to ensure each requirement is tracked through design, implementation, and testing phases.

Interface requirements

Interface requirements in software engineering specify how different components of a system, or how the system interacts with users and other systems. These requirements ensure seamless communication and functionality, both within the software itself and with external entities. Here's a detailed overview of interface requirements, their types, significance, and best practices.

- Definition**

- Interface Requirements** define the interactions between the system and external entities (users, hardware, or other software systems). They specify the inputs, outputs, protocols, and data formats needed for effective communication.

Types of Interface Requirements

1. User Interface Requirements:

- Focus on how users will interact with the system.
- Include layout, navigation, and accessibility guidelines.
- Example: “The main dashboard shall display user statistics in a graph format with drill-down options.”

2. Application Programming Interface (API) Requirements:

- Detail how different software components or systems interact programmatically.
- Specify endpoints, request/response formats, authentication methods, and error handling.
- Example: “The API shall return user data in JSON format, and support GET and POST methods.”

3. Hardware Interface Requirements:

- Describe interactions between the software and hardware components.
- Include specifications for data exchange and control signals.
- Example: “The system shall communicate with the printer via USB using the XYZ protocol.”

4. Communication Interface Requirements:

- Outline the protocols and methods used for data exchange between systems.
- Include network requirements and data transmission standards.
- Example: “Data shall be transmitted over HTTPS using TLS encryption.”

5. Integration Requirements:

- Define how the system integrates with other systems or services.
- Include details on data formats, synchronization, and error handling.

Example: “The system shall integrate with the CRM API to sync user data every hour.”

Importance of Interface Requirements

- **Clarity in Interaction:** Ensure that all stakeholders understand how different parts of the system will communicate.
- **Usability:** Well-defined user interface requirements improve the user experience, making software intuitive and accessible.
- **Interoperability:** Clear API and integration requirements facilitate smooth interactions with external systems, enhancing functionality.
- **Error Reduction:** Detailed specifications help minimize misunderstandings and errors during development and testing.

Gathering Interface Requirements

Stakeholder Interviews: Discuss needs and expectations with users and technical teams to gather insights on interface interactions.

Prototyping: Create wireframes or mockups for user interfaces to visualize interactions and gather feedback.

Documentation Review: Analyze existing systems and interfaces to identify best practices and requirements.

Workshops: Conduct collaborative sessions with stakeholders to define and prioritize interface needs.

Documenting Interface Requirements

Interface Specification Document: A comprehensive document that outlines all interface requirements, including details for user interfaces, APIs, and hardware interactions.

Use Cases: Describe specific scenarios in which users or systems will interact with the interface.

Interface Design Diagrams: Visual representations of the interface components and their interactions, which can help clarify requirements.

Requirements measures

Property	Measure
Speed	Processed transactions/second User/Event response time Screen refresh time
Size	K Bytes Number of RAM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Software Requirements Document (SRD)

A Software Requirements Document (SRD) is a crucial artifact in software engineering that outlines the specifications for a software system. It serves as a foundation for design, development, and testing, ensuring that all stakeholders have a clear understanding of what the software must achieve. Here's an overview of what an SRD typically includes, its structure, and best practices for creating one.

- Definition**

- Software Requirements Document (SRD)** is a comprehensive document that captures both functional and non-functional requirements of a software system. It communicates the needs and expectations of stakeholders, including users, developers, and project managers.

Purpose of an SRD

- **Clarity:** Provides a clear understanding of what the software is expected to do.
- **Guidance:** Serves as a reference throughout the software development lifecycle.
- **Agreement:** Ensures alignment among stakeholders on project goals and requirements.
- **Validation:** Forms the basis for testing and verification to ensure the final product meets its intended purpose.

Typical Structure of an SRD

1. Introduction:

- Purpose of the document.
- Scope of the software system.
- Definitions and acronyms.
- References to related documents.

2. Overall Description:

- Product perspective: How the software fits within the larger system or project.
 - Product functions: High-level description of major functions.
 - User classes and characteristics: Overview of different user types and their needs.
 - Constraints: Any limitations that might affect design and implementation.
- .

3. Functional Requirements:

- Detailed description of features and functions the system must provide.
- Organized by use cases or user stories, including inputs, processes, and outputs.
- Acceptance criteria for each requirement

4. Non-Functional Requirements:

- Quality attributes such as performance, security, usability, reliability, and maintainability.
- Constraints related to compliance, standards, and interoperability.

5. Interface Requirements:

- User interface requirements: Layout, navigation, accessibility.
- API requirements: Endpoints, data formats, protocols.
- Hardware and communication interface requirements.

6. System Features:

- Breakdown of features, including detailed descriptions and priority levels.
- Use cases or user stories for each feature.

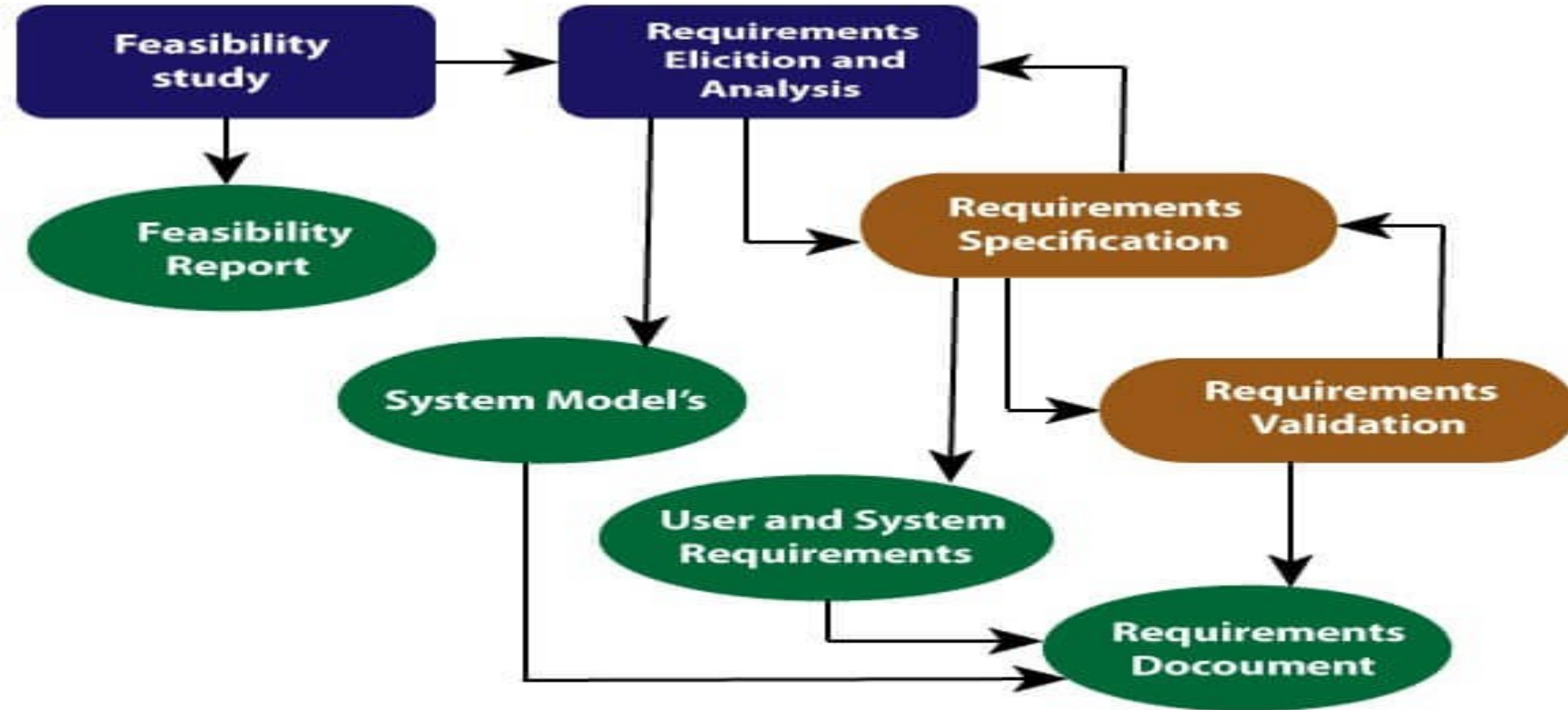
7. Appendices:

- Glossary of terms.
- Supporting diagrams (e.g., flowcharts, wireframes).
- Additional information relevant to the requirements.

Requirement Engineering Process

It is a four-step process, which includes -

1. Feasibility Study
2. Requirement Elicitation and Analysis
3. Software Requirement Specification
4. Software Requirement Validation



Requirement Engineering Process

- 1. Feasibility Study:

- The objective behind the feasibility study is to create the reasons for developing the software that is acceptable to users, flexible to change, and conformable to established standards.

- Types of Feasibility:

1. Technical Feasibility - Technical feasibility evaluates the current technologies, which are needed to accomplish customer requirements within the time and budget.

2. Operational Feasibility - Operational feasibility assesses the range in which the required software performs a series of levels to solve business problems and customer requirements.

3. Economic Feasibility - Economic feasibility decides whether the necessary software can generate financial profits for an organization.

•2. Requirement Elicitation and Analysis:

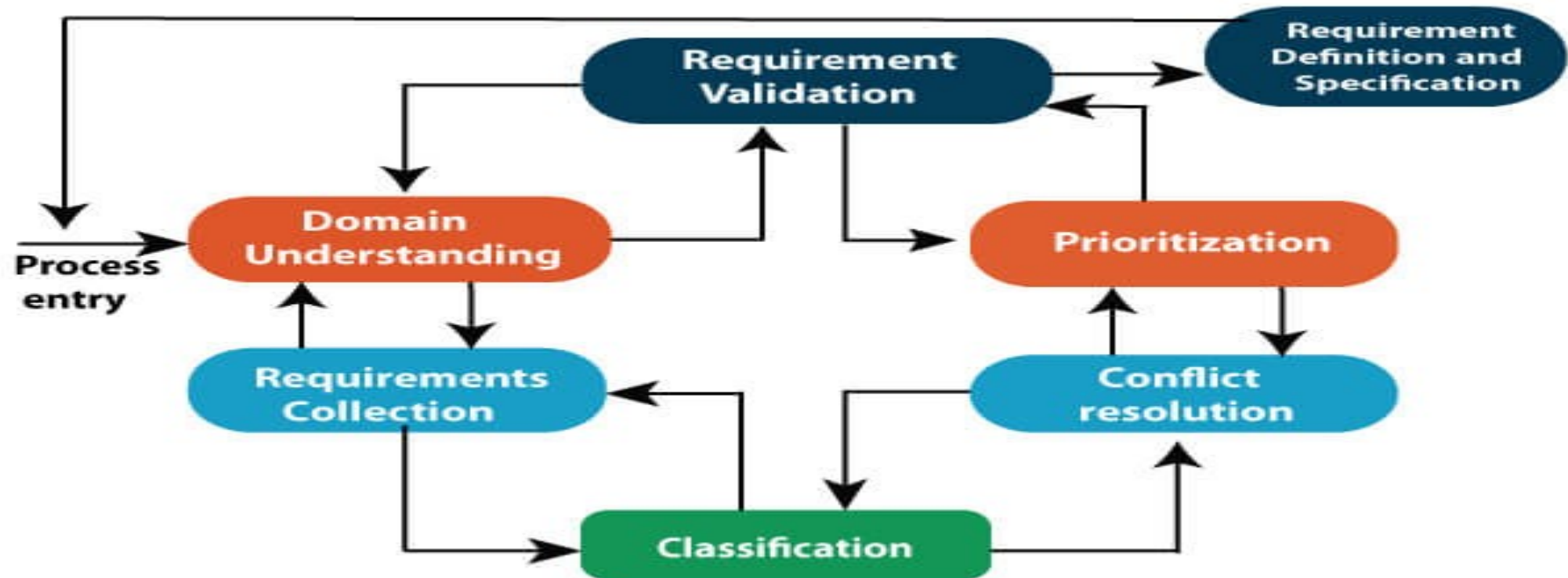
This is also known as the **gathering of requirements**. Here, requirements are identified with the help of customers and existing systems processes, if available.

Analysis of requirements starts with requirement elicitation. The requirements are analyzed to identify inconsistencies, defects, omission, etc. We describe requirements in terms of relationships and also resolve conflicts if any.

•**Problems of Elicitation and Analysis**

- Getting all, and only, the right people involved.
- Stakeholders often don't know what they want
- Stakeholders express requirements in their terms.
- Stakeholders may have conflicting requirements.
- Requirement change during the analysis process.
- Organizational and political factors may influence system requirements.

Elicitation and Analysis Process



•3. Software Requirement Specification:

- Software requirement specification is a kind of document which is created by a software analyst after the requirements collected from the various sources - the requirement received by the customer written in ordinary language. It is the job of the analyst to write the requirement in technical language so that they can be understood and beneficial by the development team.
- The models used at this stage include ER diagrams, data flow diagrams (DFDs), function decomposition diagrams (FDDs), data dictionaries, etc.
 - **Data Flow Diagrams:** Data Flow Diagrams (DFDs) are used widely for modeling the requirements. DFD shows the flow of data through a system. The system may be a company, an organization, a set of procedures, a computer hardware system, a software system, or any combination of the preceding. The DFD is also known as a data flow graph or bubble chart.

- Data Dictionaries: Data Dictionaries are simply repositories to store information about all data items defined in DFDs. At the requirements stage, the data dictionary should at least define customer data items, to ensure that the customer and developers use the same definition and terminologies.
- Entity-Relationship Diagrams: Another tool for requirement specification is the entity-relationship diagram, often called an "*E-R diagram*." It is a detailed logical representation of the data for the organization and uses three main constructs i.e. data entities, relationships, and their associated attributes.

4. Software Requirement Validation:

•After requirement specifications developed, the requirements discussed in this document are validated. The user might demand illegal, impossible solution or experts may misinterpret the needs. Requirements can be the check against the following conditions -

1. If they can practically implement
2. If they are correct and as per the functionality and specially of software
3. If there are any ambiguities
4. If they are full
5. If they can describe

Requirements Validation Techniques

- 1. Requirements reviews/inspections:** systematic manual analysis of the requirements.
- 2. Prototyping:** Using an executable model of the system to check requirements.
- 3. Test-case generation:** Developing tests for requirements to check testability.
- 4. Automated consistency analysis:** checking for the consistency of structured requirements descriptions.